🚀 Reporte Final de QA - Rama `develop` (SGI-U)

Fecha de Ejecución: 22 de Septiembre de 2026

1. Resumen de Pruebas de Backend

Responsable: Lucrecia (Lógica de Negocio y Backend)

- **Comando Ejecutado:** `./mvnw clean test` (Perfil H2 en memoria)
- **Resultados de Ejecución: 115 Tests Exitosos** (0 fallos, 0 errores).
- **Cobertura Confirmada:**
  - ✅ **Validación de DTOs:** Los decoradores `@Positive`, `@NotBlank`, etc., están funcionando correctamente y rechazando inputs inválidos.
  - ✅ **Concurrencia de Inventario:** El decremento atómico en el stock durante ventas simultáneas está respaldado y funcionando sin condiciones de carrera.
  - ✅ **Cálculos Financieros:** El recálculo recíproco de costos, márgenes y ganancias pasa todos los casos de prueba unitarios.

2. Resumen de Pruebas de Frontend

Responsale original: Leo / Lucrecia

- **Comando Ejecutado (Linter):** `flutter analyze`
- **Comando Ejecutado (Testing):** `flutter test`

Evolución del Análisis Estático (Linter)

Inicialmente, el análisis arrojó **137 issues** (Advertencias y propiedades deprecadas). Se procedió a limpiar toda la deuda técnica mediante commits atómicos:

1. **Propiedades deprecadas (UI):** Se actualizó `RadioListTile` (usando `RadioGroup`) y se migró `value` a `initialValue` en `DropdownButtonFormField`.
2. **Crash prevention (Async Gaps):** Se añadieron comprobaciones `if (!context.mounted) return;` para evitar llamadas a un context que ya no existe tras un `await` o un `Future.delayed`.
3. **Limpieza de sintaxis:** Se aplicó `dart fix --apply` para colocar `const` y llaves `{}` faltantes de forma masiva, y se corrigieron `getters`/`setters` redundantes en la clase `ApiService`.

- **Resultado Final Linter:** `0 issues found.` ✅

Resultados de Ejecución de Pruebas

- **Resultados de Ejecución: 181 Tests Exitosos** (100% Passed) en ~42 segundos.
- **Cobertura Confirmada:**
  - ✅ **Tests de Estado (Bloc):** Manejo de autenticación, finanzas y cálculo de totales en el carrito.
  - ✅ **Tests de Widgets:** Flujos modales (formularios de productos y movimientos) y pantallas principales renderizan y responden correctamente a la interacción sin romperse tras las refactorizaciones.

3. Recomendaciones y Propuestas de Mejora para Frontend (Lucrecia/Leo)

Dado el análisis y las correcciones realizadas, se proponen las siguientes prácticas para el equipo de frontend:

1. **Uso riguroso de `context.mounted`:** Nunca usar `BuildContext` (ej. navegaciones, `ScaffoldMessenger`, invocaciones a `Bloc`) después de un `await` sin antes validar `if (!context.mounted) return;`. Esto previene crashes silenciosos y excepciones de ciclo de vida cuando un usuario cierra un modal antes de que la petición HTTP finalice.
2. **Revisión continua de widgets deprecados:** Flutter evoluciona rápido. Prestar atención a los avisos de desuso (deprecation warnings) en la consola durante el desarrollo. Actualizar a las nuevas APIs inmediatamente, como sucedió con `RadioListTile` y los modales.
3. **Integración Continua (CI) para el Linter:** Se acumuló una gran cantidad de deuda (137 advertencias). Es altamente recomendable configurar un *pre-commit hook* (con `husky` o `lefthook`) o un pipeline de CI (ej. GitHub Actions) que corra `flutter analyze` y bloquee los PRs si hay issues pendientes. Esto forzará al equipo a mantener el código limpio en el día a día.
4. **Uso de `dart fix` en el entorno local:** Acostumbrarse a correr `dart fix --apply` periódicamente antes de commitear o activar la funcionalidad automática de corrección (Fix All On Save) en VSCode o Android Studio.

4. Veredicto Final

🟢 **Estado de la Rama `develop`: ESTABLE (VERDE)**.

Tanto el backend como el frontend están compilando limpiamente, pasando el 100% de su suite de tests (296 tests combinados), y el código frontend ha sido desprovisto de advertencias, bugs de async gaps y código obsoleto. La integración está lista para pasar a la siguiente fase o ser enviada a `main` (Release).